

Interpolación por Diferencias Divididas de Newton

Implementación simbólica en GNU Octave y análisis de resultados

1 de junio de 2026

1 Introducción

Se construye el polinomio que interpola a la función

$$f(x) = \log_x(\arcsin(x)) = \frac{\ln(\arcsin(x))}{\ln(x)},$$

en los nodos equiespaciados $x_i = 0.1, 0.2, \dots, 0.8$ del intervalo $[0.1, 0.8]$, empleando el *método de diferencias divididas de Newton*. La implementación se realizó en GNU Octave mediante la función `ddn(x, y)`, que sigue el pseudocódigo de la versión simbólica del documento de referencia: construye la tabla de diferencias divididas, obtiene los coeficientes de Newton y genera el polinomio como una *expresión simbólica*.

2 Estrategia de implementación

La estrategia reproduce directamente la definición recursiva del método y se apoya en el paquete simbólico de Octave (`pkg load symbolic`), de modo que el polinomio se obtiene como una fórmula algebraica en una variable simbólica X , y no solo como un conjunto de valores numéricos. El procedimiento consta de dos pasos:

1. **Tabla de diferencias divididas.** Se inicializa la primera columna con los valores y_i (diferencias de orden cero, $f[x_i] = y_i$) y se completan las columnas siguientes con la recurrencia

$$f[x_i, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}.$$

2. **Construcción simbólica del polinomio.** Partiendo del primer coeficiente $f[x_0]$, se acumula un término producto $(X - x_0)(X - x_1) \cdots$ y se suma cada coeficiente multiplicado por dicho término, tal como indica el pseudocódigo. Finalmente se expande la expresión con `expand` para obtener el polinomio en potencias de X .

El grado del polinomio se determina automáticamente como $n = \text{length}(\mathbf{x}) - 1$, por lo que la misma función sirve para cualquier cantidad de nodos.

3 Construcción de la tabla de diferencias divididas

La tabla es triangular: cada columna tiene una entrada menos que la anterior. La columna de orden 0 contiene los valores y_i ; cada entrada de la columna de orden k se obtiene a partir de dos entradas contiguas de la columna anterior, divididas por la diferencia de los nodos extremos. La Tabla 1 muestra el resultado obtenido por el programa.

Los coeficientes del polinomio de Newton corresponden a la *primera fila* de la tabla, es decir, a las diferencias divididas $f[x_0, \dots, x_k]$.

x_i	orden 0	orden 1	orden 2	orden 3	orden 4	orden 5	orden 6	orden 7
0.1000	0.9993	-0.0348	-0.2605	-0.6984	-1.6157	-4.5995	-16.6375	-89.5151
0.2000	0.9958	-0.0869	-0.4700	-1.3447	-3.9155	-14.5820	-79.2981	
0.3000	0.9871	-0.1809	-0.8734	-2.9109	-11.2065	-62.1609		
0.4000	0.9690	-0.3555	-1.7466	-7.3935	-42.2869			
0.5000	0.9335	-0.7049	-3.9647	-24.3082				
0.6000	0.8630	-1.4978	-11.2572					
0.7000	0.7132	-3.7492						
0.8000	0.3383							

Cuadro 1: Tabla de diferencias divididas generada por la función `ddn`.

4 Polinomio interpolador obtenido

Con los ocho nodos se obtiene un polinomio de grado 7. Sus coeficientes en la forma de Newton son:

$$\begin{aligned}
c_0 &= 0.999274 \\
c_1 &= -0.0347781 \\
c_2 &= -0.260475 \\
c_3 &= -0.698383 \\
c_4 &= -1.61571 \\
c_5 &= -4.59951 \\
c_6 &= -16.6375 \\
c_7 &= -89.5151
\end{aligned}$$

de manera que el polinomio en la forma de Newton es

$$P_7(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + \cdots + c_7(x - x_0)(x - x_1) \cdots (x - x_6).$$

Tras expandirlo simbólicamente con `expand`, el polinomio en potencias de x resulta

$$\begin{aligned}
P_7(x) = & -89.5234 x^7 + 234.031 x^6 - 257.875 x^5 + 151.594 x^4 \\
& - 51.3594 x^3 + 9.67969 x^2 - 0.954956 x + 1.03651.
\end{aligned}$$

5 Gráfica comparativa

La Figura 1 superpone los datos originales, la función exacta y el polinomio interpolador en el intervalo $[0.1, 0.8]$.

6 Comandos y funciones de GNU Octave

Los principales elementos del lenguaje empleados fueron:

- `pkg load symbolic`: carga el paquete simbólico, necesario para declarar variables X y operar con expresiones algebraicas.
- `@(t) log(asin(t))./log(t)`: función anónima que define f ; `asin` es el arcoseno y `log` el logaritmo natural, con los que se implementa el cambio de base de $\log_x$.
- `x = 0.1:0.1:0.8`: genera el vector de nodos equiespaciados.
- `sym` y `syms`: convierten los datos y declaran la variable X como símbolos para el cálculo exacto.

Interpolación por Diferencias Divididas de Newton

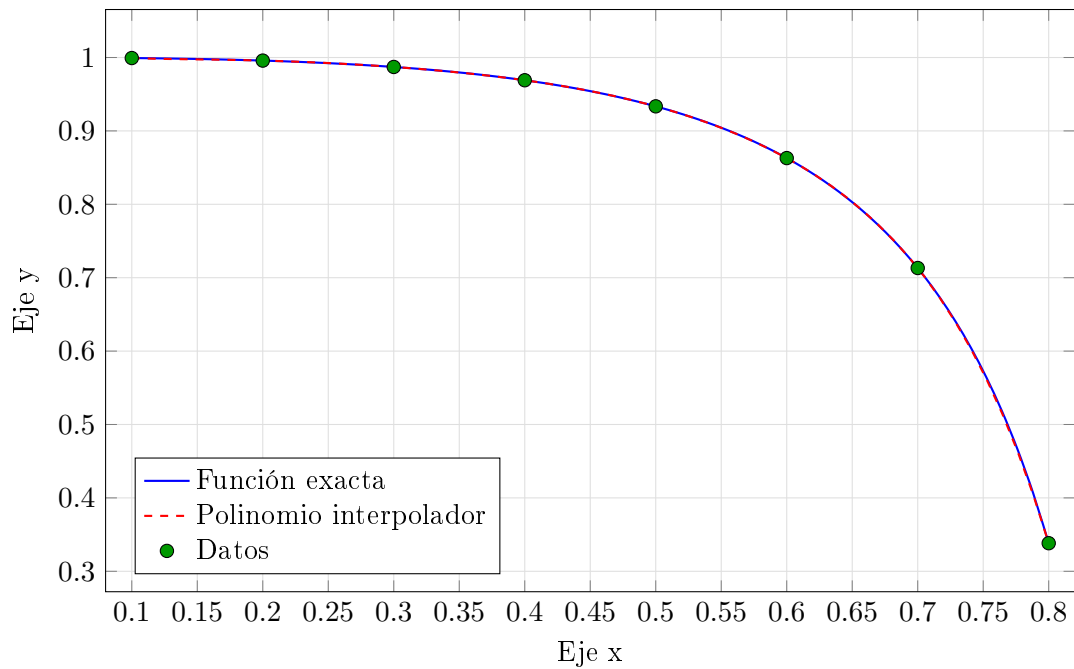


Figura 1: Datos, función exacta y polinomio interpolador de Newton en $[0.1, 0.8]$.

- Bucles **for** anidados: rellenan la tabla aplicando la recurrencia de las diferencias divididas.
- **expand**: expande el producto de factores para escribir el polinomio en potencias de x .
- **double** y **vpa**: convierten las expresiones simbólicas a forma numérica (**double**) o a punto flotante de precisión arbitraria y legible (**vpa**).
- **matlabFunction(P)**: convierte el polinomio simbólico en una función evaluable para graficarlo sobre una malla fina.
- **linspace**, **plot**, **hold on**, **grid on**, **title**, **xlabel**, **ylabel** y **legend**: construyen la figura comparativa con título, ejes etiquetados y leyenda.
- **printf**, **disp** y **diary**: imprimen en pantalla y guardan la salida en un archivo de texto.

El núcleo del cálculo de la tabla y del polinomio es:

```

1  tabla = sym(zeros(n+1, n+1));
2  tabla(:, 1) = y;                % orden 0: f[xi] = yi
3  for j = 2:(n+1)
4      for i = 1:(n+2-j)
5          tabla(i, j) = ( tabla(i+1, j-1) - tabla(i, j-1) ) ...
6                          / ( x(i+j-1) - x(i) );
7      end
8  end
9  coef = tabla(1, :);              % coeficientes de Newton
10 syms X
11 polinomio = coef(1); termino = sym(1);
12 for k = 1:n
13     termino = termino * (X - x(k));
14     polinomio = polinomio + coef(k+1) * termino;
15 end
16 P = expand(polinomio);            % polinomio simbolico

```

7 Comportamiento del polinomio y precisión

El polinomio interpolante reproduce los ocho datos: el error máximo medido en los nodos, $\max_i |P_7(x_i) - y_i|$, fue del orden de $1.570e - 07$, pequeño y acorde con la aproximación racional que el paquete simbólico aplica a los valores de f al convertirlos en símbolos.

Entre los nodos, el polinomio sigue muy de cerca a la función exacta en casi todo el intervalo, como se observa en la Figura 1, donde ambas curvas resultan prácticamente indistinguibles. La región más exigente es la cercana a $x = 0.8$: allí la función cae con gran rapidez y los coeficientes de orden alto de la tabla crecen en magnitud (hasta cerca de -89.5), lo que refleja la fuerte curvatura de f . Aun así, dentro de $[0.1, 0.8]$ la aproximación es satisfactoria y no se aprecian oscilaciones marcadas tipo Runge, gracias a que el intervalo es estrecho y los nodos cubren bien la región de interés.

En conclusión, la implementación simbólica del método de diferencias divididas de Newton interpola los datos suministrados y aproxima de forma fiable la función en el intervalo considerado, entregando además el polinomio como una expresión algebraica explícita.